

Advanced Time Series - Computer Exercises 4

Authors

Gunn Jacobsen - s175799
Linda Elisabet á Geilini - s223116

Group: 9

September 18, 2026

Contents

1 Introduction **1**

2 Part 1 - AR Model **1**

3 Part 2 - TARX Model **3**

4 Part 3 - Include Diurnal Variations **6**
4.1 1-hour Predictions 6
4.2 2-hour Predictions 8
4.3 3-hour Predictions 10

5 Part 4 - Comments/Discussion **12**

6 Appendex A **13**
6.1 Code 13

1 Introduction

In this final computer exercise, we have chosen to work on forecasting wind power. The aim of this exercise is to develop adaptive models for predicting wind power 1, 2, and 3 hours ahead. The provided dataset comes from Klim, a wind farm in Northwest Jutland. We will only consider the previously processed and interpolated data. The available observations in the dataset include hourly time intervals, average wind power, and one-, two-, and three-hour weather forecasts, which include wind speed, wind direction, and temperature. For simplicity, we will assume that the weather forecasts are accurate and free from uncertainty.

In Figure 1, data from the first week of January 1999 is plotted, showing wind direction, power, and speed. It is clear that there is a positive correlation between wind speed and wind power. The wind direction observations range from 0 to 360 degrees, where 0 corresponds to wind coming from the north, 90 from the east, 180 from the south, and 270 from the west. A closer look at Figure 1 reveals that higher wind speeds are more frequently observed when the wind is coming from the west. This is approximately when direction readings fall between 200 and 300 degrees. Considering the location of the wind turbines, this observation aligns with expectations, as west is the prevailing wind direction in Denmark.

We will take these observations into account when developing the adaptive models. Additionally, we will consider the fact that diurnal variation is commonly observed due to the daily heating and cooling of the Earth's surface. To evaluate the models quantitatively, we will use Root Mean Squared Error (RMSE) as the performance metric along with some diagnostic plots of the residuals.

We will start by focusing on modeling the one-hour-ahead forecast, iteratively improving this model until we are satisfied with its performance. Once the one-hour-ahead model is finalized, we will continue to develop models for the two- and three-hour-ahead forecasts based on the one-hour-ahead model, but where we make adjustments to account for the larger forecasting horizon. This way we reduce the number of iterative steps needed to complete the three models, as our main interests are the final models.

We split the data into a training set and a testing set, that are from different years, but the same two quarters of the year. The performance metric is calculated based on approximately the same time period for all models, but for the 2 hour forecast we only consider every other observation and for the 3-hour model we only consider every third observation.

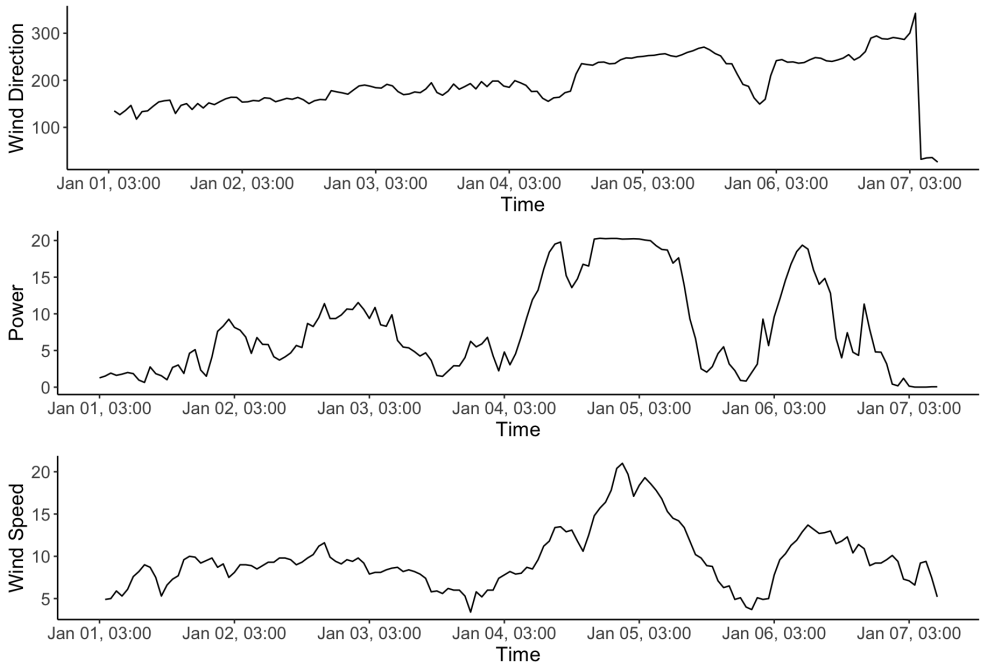


Figure 1: Time series for wind- direction, power and speed from 1 to January 7th 1999

2 Part 1 - AR Model

We begin by fitting a pure autoregressive (AR) model defined as:

$$Y_t = \left(\sum_{i=1}^{p} \phi_{t-i} Y_{t-i} \right) + \epsilon_t$$

where $\epsilon_t \sim N(0, \sigma)$ and i.i.d. To determine the appropriate number of lags p for the AR model, we examine the Partial Autocorrelation Function (PACF) of the wind power observations, as shown in Figure 2. The PACF suggests that lag 4 is significant. We then fit AR models of orders 1 through 5 and compare their AIC and BIC values, presented in Table 3. Since the $AR(4)$ model has the lowest AIC and BIC, we select this model and proceed to estimate its parameters. The parameter estimates are obtained using Conditional Sum of Squares (CSS) for initial estimates, followed by Maximum Likelihood Estimation (MLE). This approach corresponds to the method implemented by the `arima` function in R. The estimated parameters, along with their standard errors, are listed in Table 4.

Figure 5 displays the observed wind power values alongside the one-hour-ahead forecasts from the AR(4) model. As seen in the figure, the forecasts tend to be systematically higher than the actual values, indicating that the AR model alone does not capture the full dynamics of wind power generation. The RMSE of the model is 162.8

A pure AR model relies only on past observations of the series itself and does not incorporate external explanatory variables. However, wind power output is primarily driven by external factors such as wind speed, wind direction, and atmospheric conditions. Without these inputs, the AR model cannot accurately represent the underlying state of the system. In addition, wind patterns vary over time due to seasonal effects and changing atmospheric conditions. A pure AR model assumes stationarity, meaning the mean and variance remain constant over time. Since the mean and variance of wind power are heavily dependent on wind speed, this assumption is not reasonable.

Models	AIC	BIC
$AR(1)$	12055.7	12073.7
$AR(2)$	11989.6	12013.6
$AR(3)$	11988.1	12018.2
$AR(4)$	11966.2	12002.2
$AR(5)$	11967.5	12009.5

Figure 3: AIC and BIC for auto regressive models of different order

Parameters	Estimate	Variance
Intercept	5.69	2.7e-01
ϕ_1	1.10	3.3e-04
ϕ_2	-0.20	7.3e-04
ϕ_3	0.13	7.3e-04
ϕ_4	-0.09	3.3e-04

Figure 4: Estimated parameters of a $AR(4)$ process

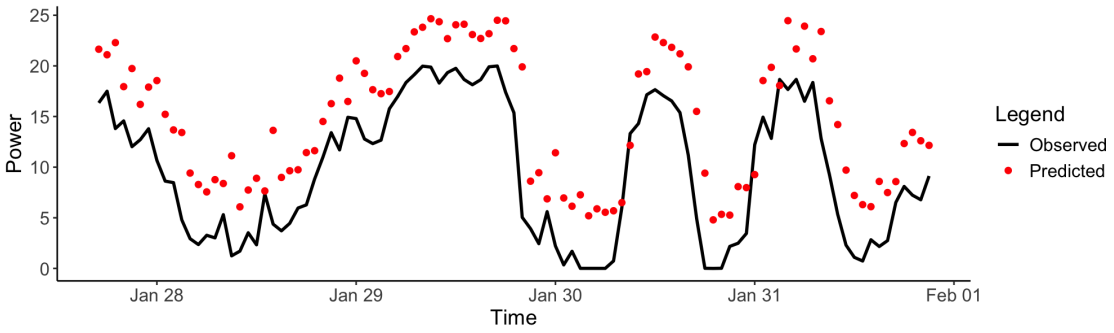


Figure 5: Caption

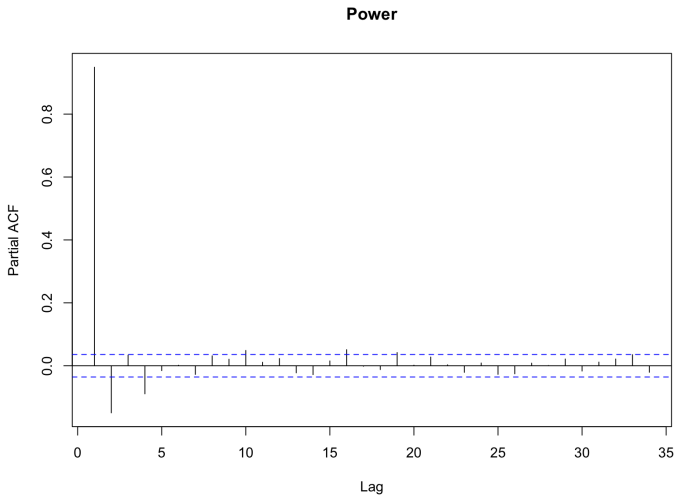


Figure 2: Partial Autocorrelation Function for the wind power observtaions

3 Part 2 - TARX Model

We saw in the previous section that a pure autoregressive model didn't perform so well, when using it to predict future wind power. Intuitively, we know that the power production shouldn't only be affected by previous values, but should also be influenced by the wind speed, which we also saw by the positive correlation between wind speed and wind power in Figure 1. Furthermore we saw that different wind directions had different wind powers. We will therefore extend the model by including a power curve term, that varies depending on the wind direction.

Each observation includes a wind direction value provided as a numerical degree between 0 and 360, where 0 indicates wind blowing directly from the north, 90 directly from the east, 180 directly from the south, and 270 directly from the west. We will split the total 360 degrees into four intervals, each corresponding to one of the four wind directions. The wind directions are defined as:

$$\begin{aligned} North &= [315, 360[\cup[0, 45[\\ East &= [45, 135[\\ South &= [135, 225[\\ West &= [225, 315[\end{aligned}$$

For each wind direction, we will fit the power curve in R as a sigmoid function scaled by the maximum wind power in that direction:

$$PC_{wd} = \frac{\max(windpower_{wd})}{1 + e^{-\alpha(ws-\beta)}}$$

Where ws is the wind speed. The fitted curves are displayed in Figure 6. The parameter estimates for each curve are obtained by splitting the training dataset into four subsets, one for each wind direction. Each subset contains observations of wind speed and power corresponding to a specific direction. The parameters are estimated by minimizing the sum of squared errors (SSE). The minimization of SSE is achieved using the `optim()` function in R with the BFGS method. We find the different curves to be as follows:

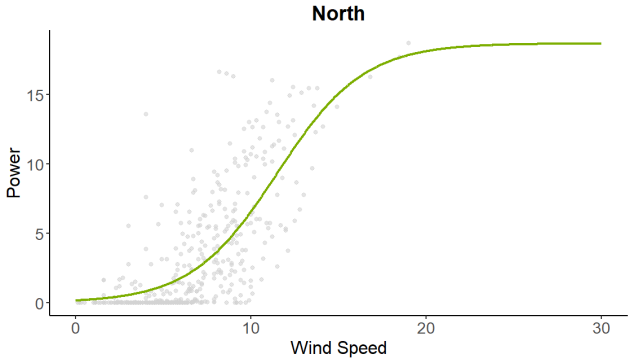
$$PC_N = \frac{18.69}{1 + e^{-0.41(ws-11.50)}} \quad PC_E = \frac{19.66}{1 + e^{-0.42(ws-11.73)}}$$
$$PC_S = \frac{20.3}{1 + e^{-0.45(ws-10.22)}} \quad PC_W = \frac{20.3}{1 + e^{-0.35(ws-11.36)}}$$

The different power curves display slight variations in shape, supporting the idea that wind direction influences wind power generation. The power curve for the South wind direction is the steepest and appears to reach the highest maximum power. The West wind direction reaches a similar maximum power but follows a less steep progression. The power curves for East and North closely follow each other in shape, in the beginning, though they converge to different maximum power values, with the North wind direction reaching a lower maximum than the East.

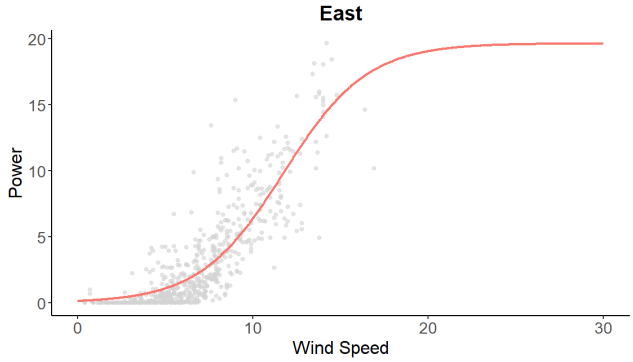
We now want to extend the autoregressive model by including the power curve as an external input. As the power curve is different for the different wind directions, we will have four regimes, where the wind direction determines the switch between regimes. In the previous section, we found that four lags were significant. Including four lags and an external input for four different regimes leads to us having to estimate 20 different parameters. We know that having a high number of parameters can lead to overfitting and reduced reliability of optimization functions like `optim()`. We therefore choose not to include the fourth lag in order to reduce the number of estimated parameters to 16.

The resulting TARX(3,4) model can be expressed as follows.

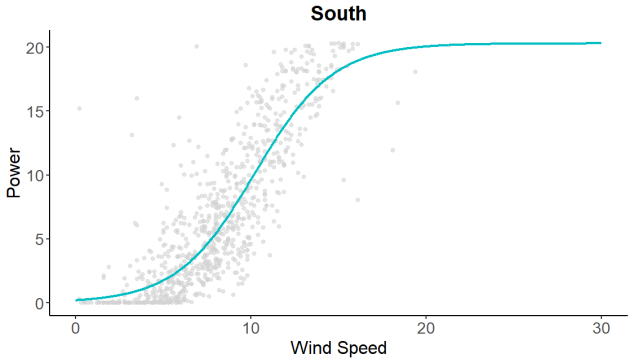
$$X_t = \begin{cases} a_{11}X_{t-1} + a_{12}X_{t-2} + a_{13}X_{t-3} + b_1 PC_N(ws, wd) + \epsilon_t & \text{if } wd \in \text{North} \\ a_{21}X_{t-1} + a_{22}X_{t-2} + a_{23}X_{t-3} + b_2 PC_E(ws, wd) + \epsilon_t & \text{if } wd \in \text{East} \\ a_{31}X_{t-1} + a_{32}X_{t-2} + a_{33}X_{t-3} + b_3 PC_S(ws, wd) + \epsilon_t & \text{if } wd \in \text{South} \\ a_{41}X_{t-1} + a_{42}X_{t-2} + a_{43}X_{t-3} + b_4 PC_W(ws, wd) + \epsilon_t & \text{if } wd \in \text{West} \end{cases} \tag{1}$$



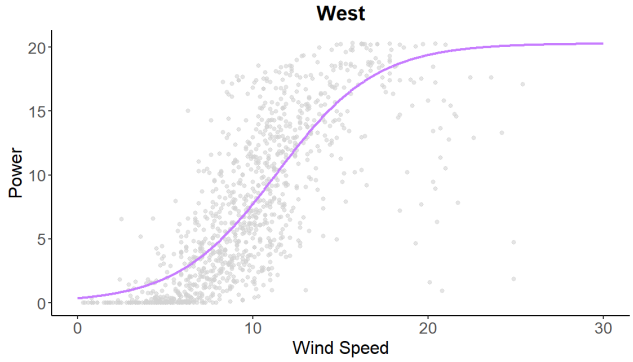
(a)



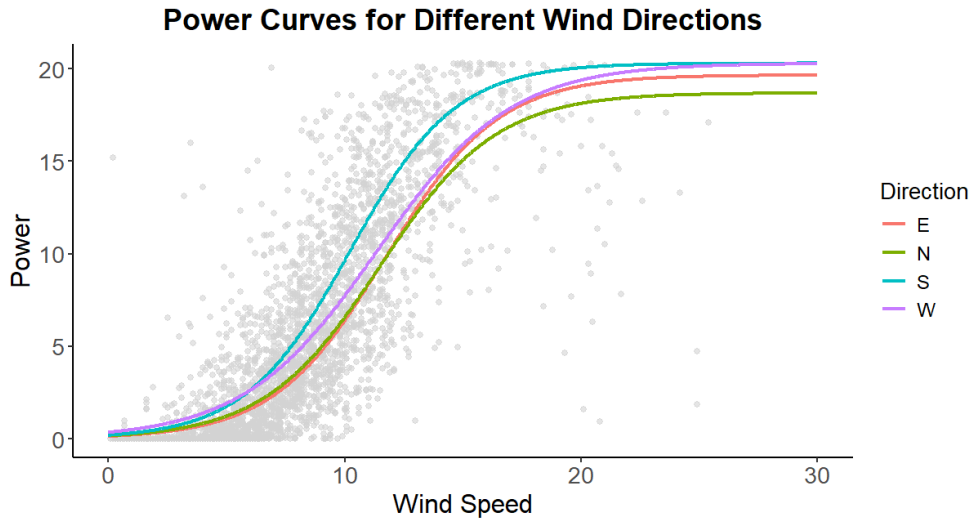
(b)



(c)



(d)



(e)

Figure 6: Power curves for different wind directions

The model in (1) is fitted in R on the training set. The parameters are again found using the prediction error method, and they are displayed in Table 1.

Parameter	North	East	South	West
$\hat{a}_1$	1.18	1.16	0.94	1.01
$\hat{a}_2$	-0.32	-0.40	-0.8	-0.19
$\hat{a}_3$	0.01	0.10	-0.7	-0.01
$\hat{b}$	0.11	0.24	0.21	0.17

Table 1: TARX(3,4) Model Parameters

We now perform one-hour-a-head predictions on the first 1000 data points in the chosen test set. Like before the choose to focus on the period from 17:00, January 27th, year 2000 to 21:00, January 31th, year 2000. The predictions are displayed in Figure 7.

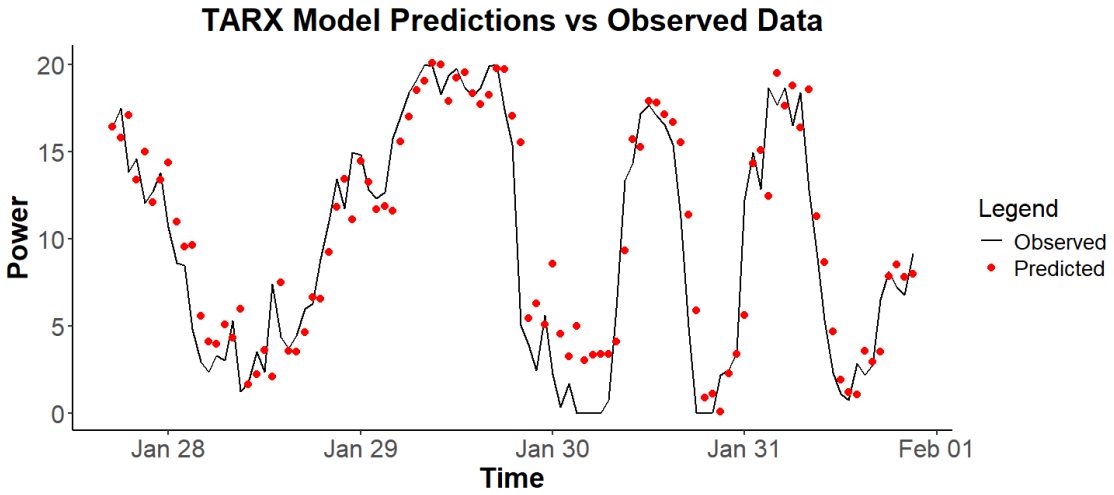


Figure 7: Predictions for 17:00, January 27th, year 2000 to 21:00, January 31th, year 2000.

.

Based on the plot we can see that there is a significant improvement from the purely autoregressive model to the TARX model. The predictions seem to follow the true data much better and do not display the same tendency to constantly overestimate the wind power.
There does appear to be slight overestimation when the wind power is decreasing and slight underestimation when the wind power is increasing. This suggests that the model isn't able to accurately adapt to the changes in wind power and seems to follow behind the actual wind power. We also see that the model has a larger overestimation around January 30. From studying the data, this is at a point, where the wind speed is too high for power production and the model seems to have a problem accurately capturing this change.

We calculate the RMSE for the predictions of the model on the 1000 data points in the test set and find $RMSE = 2.998$, Which is a significant improvement when compared to the RMSE of the AR models.

4 Part 3 - Include Diurnal Variations

We now want our model to account for diurnal changes. A strip of data for wind power is displayed in Figure 8.

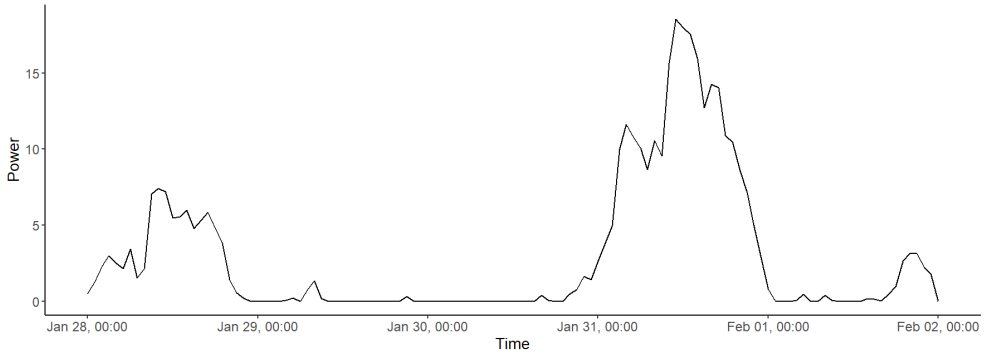


Figure 8: Plot of windpower from 00:00 January 28th 1999 to 00:00 February 1999.

Studying the plot we see some indications of diurnal changes in wind power throughout the day. Especially when considering the change in wind power from January 28th 00:00 to January 29th 00:00 and again from January 31th 00:00 to February 1st 00:00. The evolution of windpower during these days suggests that the windpower is lowest around midnight, increases during the day, and then decreases again before midnight. To account for these diurnal changes in wind power, we will include oscillatory terms that will be multiplied by the power curves in our model. Specifically we will include the term:

$$sin\left(\frac{2\pi h}{24}\right) + cos\left(\frac{2\pi h}{24}\right)$$

This term can help capture the daily oscillation in wind power and by fitting this term to the data, the corresponding parameter can ensure appropriate peaks of wind power during the day. The extended model can be expressed as:

$$X_t = \begin{cases} a_{11}X_{t-1} + a_{12}X_{t-2} + a_{13}X_{t-3} + b_1 PC_N(ws, wd)\left(1 + sin\left(\frac{2\pi h}{24}\right) + cos\left(\frac{2\pi h}{24}\right)\right) + \epsilon_t & \text{if } wd \in \text{North} \\ a_{21}X_{t-1} + a_{22}X_{t-2} + a_{23}X_{t-3} + b_2 PC_E(ws, wd)\left(1 + sin\left(\frac{2\pi h}{24}\right) + cos\left(\frac{2\pi h}{24}\right)\right) + \epsilon_t & \text{if } wd \in \text{East} \\ a_{31}X_{t-1} + a_{32}X_{t-2} + a_{33}X_{t-3} + b_3 PC_S(ws, wd)\left(1 + sin\left(\frac{2\pi h}{24}\right) + cos\left(\frac{2\pi h}{24}\right)\right) + \epsilon_t & \text{if } wd \in \text{South} \\ a_{41}X_{t-1} + a_{42}X_{t-2} + a_{43}X_{t-3} + b_4 PC_W(ws, wd)\left(1 + sin\left(\frac{2\pi h}{24}\right) + cos\left(\frac{2\pi h}{24}\right)\right) + \epsilon_t & \text{if } wd \in \text{West} \end{cases} \tag{2}$$

4.1 1-hour Predictions

We fit the model using the data for the 1-hour forecasts. The model parameters are found using the prediction error method and are displayed in Table 2.

Parameter	North	East	South	West
$\hat{a}_1$	1.24	1.18	1.04	1.09
$\hat{a}_2$	-0.34	-0.38	-0.06	-0.19
$\hat{a}_3$	0.05	0.15	-0.01	0.06
$\hat{b}$	0.01	0.03	0.01	0.02

Table 2: Parameters of diurnal model for 1-hour forecast.

We now perform one hour a head predictions again for the 1000 data points in test set. The predictions for the period from 17:00, January 27th, year 2000 to 21:00, January 31th, year 2000 are displayed in Figure 9.

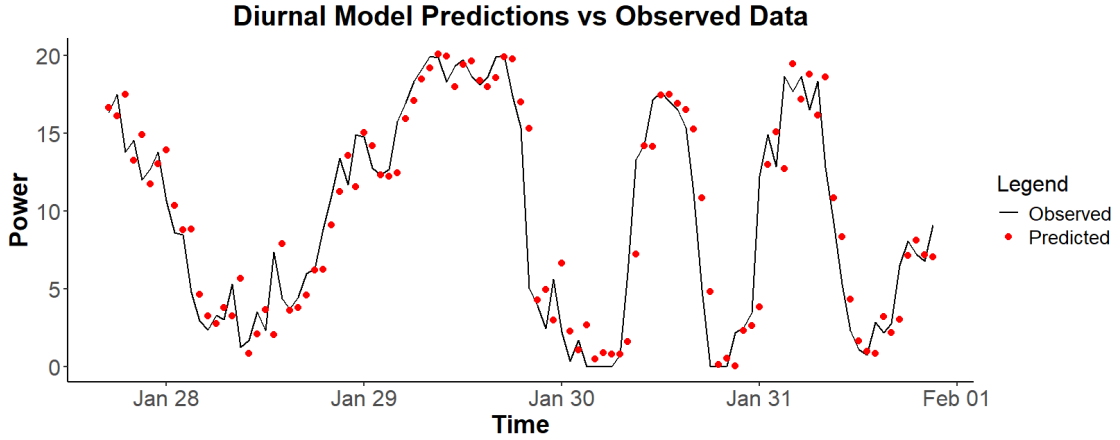


Figure 9: 1-Hour Predictions for 17:00, January 27th, year 2000 to 21:00, January 31th, year 2000.

From the plot we can see that adding the diurnal term improved the predictions. The most noticeable improvement is that the overestimation around January 30th is reduced. Furthermore, we can see that points lie closer to the observed data line than they did in Figure 7. Some even touching the line, that didn't do so with the previous model. The model still struggles with overestimation for decreasing wind power and underestimation for increasing wind power, suggesting the model still isn't able to adapt appropriately.

The RMSE for the predictions of the model on the 1000 data points in the test set is found to be: $RMSE = 0.48$, Which again is a significant improvement from the TARX model.

4.1.1 Model Diagnostic

When fitting the pure AR model, we stated the assumption that the residuals were i.i.d and normally distributed. Figure 10 presents two residual plots for the 1-hour forecast model after accounting for diurnal variations. In the left-hand plot, it is evident that the residuals do not have constant variance: when the observed power output is low, the residuals tend to be positive, and when the observed power output is high, the residuals tend to be negative. In the right-hand plot, the normality assumption also appears to be violated. The extremely high and low residuals occur more frequently than would be expected under a normal distribution.

Wind power is bounded between a lower limit of zero and an upper limit at the turbine's rated capacity. This lower and upper bound restricts the range of possible values, causing residuals to cluster near the

bounds. As a result, the distribution of the residuals is neither symmetrical nor unbounded, and their variance tends to be non-constant, thus undermining the traditional normality assumption.

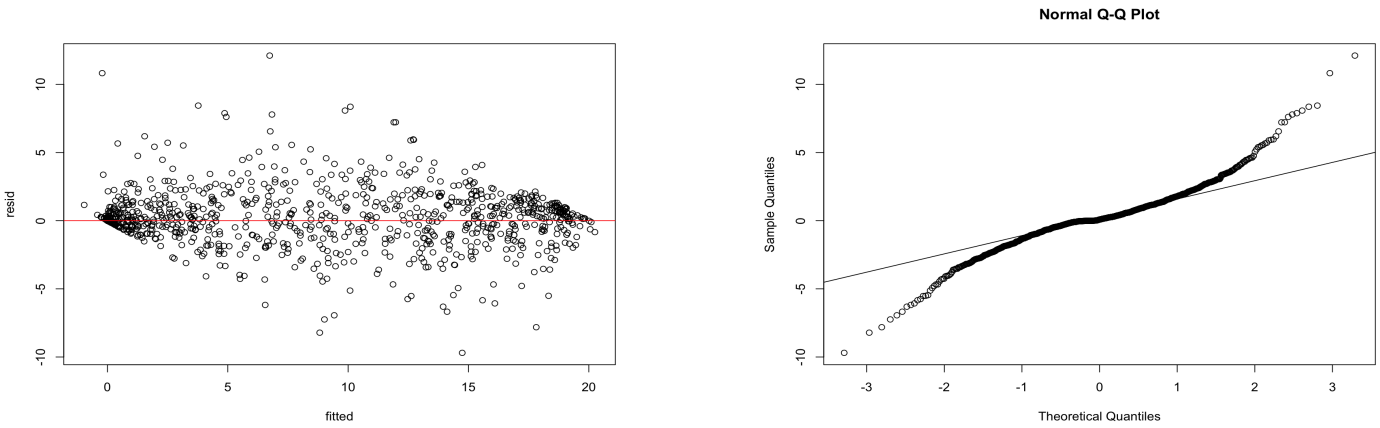


Figure 10: Diagnostics plots for 1-hour forecast model

4.2 2-hour Predictions

Until now we have primarily focused on 1-hour predictions when deciding to include additional terms to properly account for the changes in wind power. This is because the 1-hour results already demonstrate the inferiority of the simpler models, and models with longer horizons (2 and 3-hour predictions) would be even more reliant on capturing these additional complexities, making it reasonable to also include the auto-regressive, Power Curve and diurnal parts for these models. These parts might however need some adjustments, as there can be a difference in short-term and long-term dependencies.

We start by checking the number of lags needed for the 2-hour forecast model. The PACF plot is displayed in Figure 11. We can see that the third lag is barely significant, but we will choose to keep it, as a model with only 2 lags proved to perform worse.

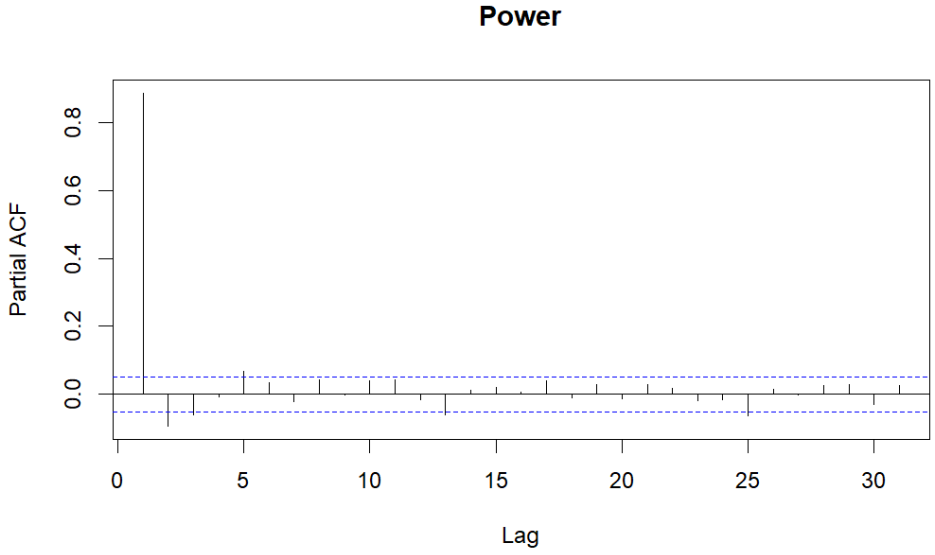


Figure 11: PACF for 2 hour data

We also need to refit the power curves to the 2 hour forcasted data as the influence of current wind speed and direction decreases because wind dynamics are more variable over longer time periods. The refited power curves are given here:

$$PC_N = \frac{17.71}{1 + e^{-0.41(ws-11.67)}} \quad PC_E = \frac{18.43}{1 + e^{-0.42(ws-11.51)}}$$
$$PC_S = \frac{20.27}{1 + e^{-0.41(ws-10.39)}} \quad PC_W = \frac{20.28}{1 + e^{-0.30(ws-11.63)}}$$

The parameters of the model are estimated with the updated power curves and they are displayed in Table 3.

Parameter	**North**	**East**	**South**	**West**
$\hat{a}_1$	1.04	0.93	1.02	0.96
$\hat{a}_2$	-0.20	-0.6	-0.04	0.01
$\hat{a}_3$	0.04	0.03	-0.06	-0.06
$\hat{b}$	0.03	0.05	0.04	0.02

Table 3: Parameters of diurnal model for 2-hour forecast.

We now perform 2-hour-ahead predictions on 500 data points in the test set. For the plot we will focus on the same period as in other sections. The plot is displayed in Figure 12. We observe longer distance between the predictions, which makes sense considering, we are only forecasting every other hour. The 2-hour predictions are less accurate than the 1-hour predictions, with larger distance between the predictions and the observed data, with only few observations touching the observed data line.

The decrease in accuracy aligns with what we would expect, because recent lags and weather conditions are not as correlated with 2-hour predictions as they are for 1-hour predictions - resulting in a decrease in predictive power. During a 2 hour period there is a higher chance of changes in external weather conditions, which increases the stochasticity of the underlying process.

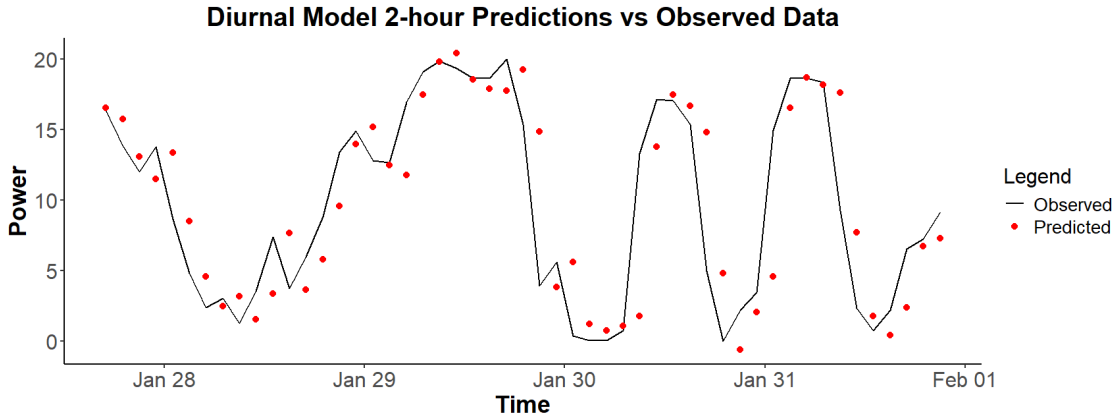


Figure 12: 2-Hour Predictions for 17:00, January 27th, year 200 to 21:00, January 31th, year 2000.

The RMSE for the predictions of the model on the 500 data points in the test set is found to be: $RMSE = 3.41$, Which again is a decrease compared to the RMSE for the 1-hour prediction model.

4.3 3-hour Predictions

We now proceed to consider 3-hour predictions. Similar to the 2-hour prediction model, we will build the 3-hour model based on the final 1-hour prediction model. As previously argued, this approach is reasonable due to the longer prediction horizons. However, it is not evident that the number of lags would remain the same, as a 3-hour prediction model with 3 lags relies on information further in the past compared to a 1-hour prediction model with 3 lags. Therefore, we begin by determining the number of lags for the 3-hour forecast model.

The PACF plot, shown in Figure 14, indicates that lag 4 is significant. However, we chose to include only 3 lags, consistent with our approach in the TARX model.

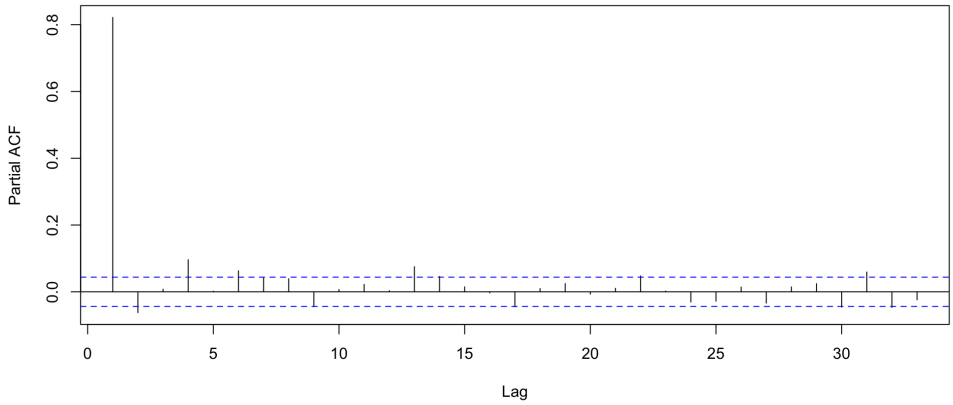


Figure 13: PACF for 3 hour data

Subsequently, we re-estimated the parameters of the power curve and the model using data with 3-hour intervals. The estimated parameters are presented in Table 4 and in equation 3 and 4. While the parameter estimates show some minor adjustments compared to the 2-hour and 1-hour forecast models, we see that it is negative in the north direction. This indicates that, when the wind is coming from the north, a reduction in power output is expected compared to other directions.

$$PC_N = \frac{17.13}{1 + e^{-0.42(ws-11.49)}} \quad PC_E = \frac{19.66}{1 + e^{-0.37(ws-12.13)}} \tag{3}$$

$$PC_S = \frac{20.24}{1 + e^{-0.39(ws-10.45)}} \quad PC_W = \frac{20.25}{1 + e^{-0.32(ws-11.9)}} \tag{4}$$

Parameter	North	East	South	West
$\hat{a}_1$	0.91	0.78	0.99	0.88
$\hat{a}_2$	-0.10	-0.04	-0.18	-0.002
$\hat{a}_3$	0.04	0.10	0.10	-0.03
$\hat{b}$	-0.02	0.07	0.04	0.04

Table 4: Model parameters for 3-hour forecast model

Figure 14 illustrates a 3-hour prediction alongside the observations. It is again clear that the model does not adapt well to the dynamics. When there is an increase in power, the model underestimates, and when there is a decrease, the model overestimates. We find that $RMSE = 6.75$, which suggests a decrease in accuracy compared to the 2-hour model.

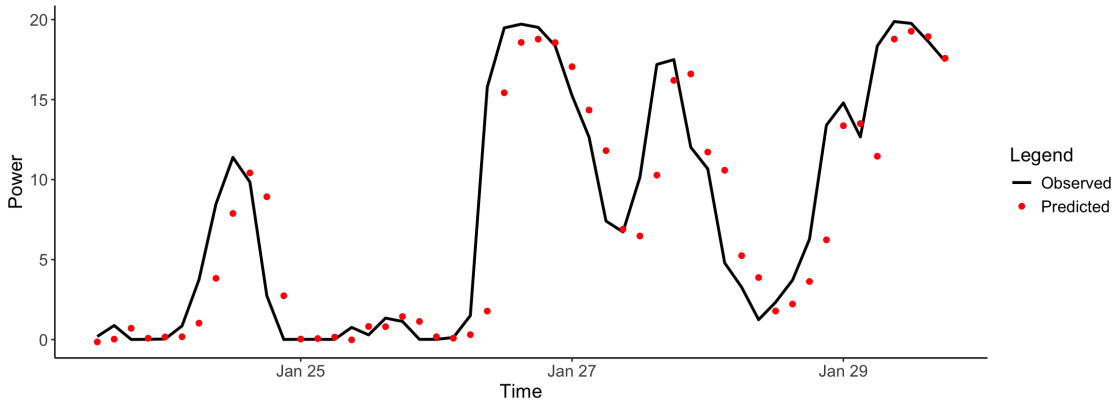


Figure 14: 3-hour predictions from 12:00, January 23th, year 2000 to 18:00, January 29th, year 2000

5 Part 4 - Comments/Discussion

Through the modeling process, we identified the model described in (2) as the most suitable to predict the wind power. The final model was able to capture the pattern of the wind power, but seemed to struggle to timely adapt to sudden changes in wind power. This could suggest that that our input signals are not strong enough. This could potentially have been improved by using more directions or by including more external inputs in our model, i.e. the effect of temperature or seasonal changes, which we didn't include. It however, is not something we will be able to implement within the time scope of the project.

We note that the final model didn't include the fourth lag, although this proved to be significant, when we fitted the pure AR model. We did try including the fourth lag in our later models, but the model performed significantly worse than the chosen model with three lags, which is the main reason we chose not include it. Especially when we know, that estimates are less reliable with more parameters and that there is a risk of overfitting.

The fitted nonlinear TARX model assumes stationarity within each regime. By adding a unique power curve for each wind direction and adding a diurnal component, we try to reduce variability and encourage stationarity. However, true stationarity may still not hold. Factors such as long-term trends (e.g., seasonal patterns over weeks or months) and other changes in turbine performance remain unmodeled.

If we had modeled the wind power using a system of SDE's we could potentially prevent the issues with the stationarity assumption. Using SDE's to model wind power would also allow us to work in continuous time, which is preferred, when modelling wind dynamics that evolve and change continuously and are affected by random fluctuations.

Restricting the model to four directional regimes is a simplified approach that may not accurately represent the continuous and complex spectrum of wind conditions. Even if we introduce more directional categories (e.g., intercardinal directions), the transitions remain abrupt. Such sharp thresholds do not capture the gradual way in which wind conditions change. For instance, two wind directions that differ by only one degree could be assigned to distinct regimes, resulting in significantly different predictions — even though in reality, the conditions are almost identical. A smoother transition between regimes would be more appropriate, ensuring that similar wind directions yield similar outputs.

6 Appendex A

6.1 Code

```
setwd("/Users/gunnjoensen/Documents/DTU/adv_time_series/Computer_Exercise4")

# import libraries ##############################################################

library(ggplot2)
library(gridExtra)
library(lubridate)
library(mgcv)


rm(list = ls())

# read data #####################################################################


X <- read.table("comp_ex_4_scripts_2011/data/cex4WindDataInterpolated.csv", sep=",",
                header=TRUE, stringsAsFactors=FALSE)
X$t <- as.POSIXct(X$t, tz="UTC")
X$hour <- hour(X$t)



# convert temperature from kelvin to Celsius
X$T1 <- X$T1 - 273.15
X$T2 <- X$T2 - 273.15
X$T3 <- X$T3 - 273.15

# add variables where wind direction is in radians
X$Wd1_radian <- (90 - X$Wd1) %% 360
X$Wd1_radian <- X$Wd1_radian*pi/180

# add variables where the forecast is shifted
X$Ws1_shift <- c(NA, X$Ws1[-nrow(X)])
X$Wd1_shift <- c(NA, X$Wd1[-nrow(X)])
X$T1_shift <- c(NA, X$T1[-nrow(X)])

X$Ws2_shift <- c(NA,NA, X$Ws2[-c(nrow(X), nrow(X) - 1)])
X$Wd2_shift <- c(NA, NA, X$Wd2[-c(nrow(X), nrow(X) - 1)])
X$T2_shift <- c(NA, NA, X$T2[-c(nrow(X), nrow(X) - 1)])

X$Ws3_shift <- c(NA, NA, NA, X$Ws3[-c(nrow(X), nrow(X) - 1, nrow(X) - 2)])
X$Wd3_shift <- c(NA, NA, NA, X$Wd3[-c(nrow(X), nrow(X) - 1, nrow(X) - 2)])
X$T3_shift <- c(NA, NA, NA, X$T3[-c(nrow(X), nrow(X) - 1,nrow(X) - 2)])

# model log p since p can not be negative
X$log.p <- log(X$p + 1e-6) # Ensure that log(0) = -Inf does not happen
```

```r
X$log.Ws1_shifted <- log(X$Ws1_shift + 1e-6)

# Pick train and test datas
X_train <- X[1:3000,]
X_test <- X[9000:10000,]
sum(is.na(X_train))
sum(is.na(X_test))
```

```r
################################################################################
## explore data
################################################################################

# data from 5 consecutive days in 2 different directions
#X_train_5Days_S <- X_train[22:142, ] # wind from south / south west
X_train_5Days_S <- X_train[1:44,] #[2710:(2710 + 120), ]
X_train_5Days_N <- X_train[1:150, ]


# Create breaks at the start of each day
break_points_S <- X_train_5Days_S$t[seq(1, nrow(X_train_5Days_S), by = 24)]
break_points_N <- X_train_5Days_N$t[seq(1, nrow(X_train_5Days_N), by = 24)]



# Plot: Wind Direction
plot_wd_S_wind <- ggplot(data = X_train_5Days_S, aes(x = t, y = Wd1_shift)) +
  geom_line() +
  labs(x = "Time", y = "Wind Direction") +
  theme_classic() +
  theme(
    axis.title = element_text(size = 14),
    axis.text = element_text(size = 12)
  ) +
  scale_x_datetime(
    breaks = break_points_S,
    date_labels = "%b %d, %H:%M"
  )

plot_wd_N_wind <- ggplot(data = X_train_5Days_N, aes(x = t, y = Wd1_shift)) +
  geom_line() +
  labs(x = "Time", y = "Wind Direction") +
  theme_classic() +
  theme(
    axis.title = element_text(size = 14),
```

```
    axis.text = element_text(size = 12)
  ) +
  scale_x_datetime(
    breaks = break_points_N,
    date_labels = "%b %d, %H:%M"
  )
```

```
# Plot: Power
plot_p_S_wind <- ggplot(data = X_train_5Days_S, aes(x = t, y = p)) +
  geom_line() +
  labs(x = "Time", y = "Power") +
  theme_classic() +
  theme(
    axis.title = element_text(size = 14),
    axis.text = element_text(size = 12)
  ) +
  scale_x_datetime(
    breaks = break_points_S,
    date_labels = "%b %d, %H:%M"
  )

plot_p_N_wind <- ggplot(data = X_train_5Days_N, aes(x = t, y = p)) +
  geom_line() +
  labs(x = "Time", y = "Power") +
  theme_classic() +
  theme(
    axis.title = element_text(size = 14),
    axis.text = element_text(size = 12)
  ) +
  scale_x_datetime(
    breaks = break_points_N,
    date_labels = "%b %d, %H:%M"
  )
```

```
# Plot: Wind Speed
plot_ws_S_wind <- ggplot(data = X_train_5Days_S, aes(x = t, y = Ws1_shift)) +
  geom_line() +
  labs(x = "Time", y = "Wind Speed") +
  theme_classic() +
  theme(
    axis.title = element_text(size = 14),
    axis.text = element_text(size = 12)
  ) +
  scale_x_datetime(
    breaks = break_points_S,
    date_labels = "%b %d, %H:%M"
```

```
  )

plot_ws_N_wind <- ggplot(data = X_train_5Days_N, aes(x = t, y = Ws1_shift)) +
  geom_line() +
  labs(x = "Time", y = "Wind Speed") +
  theme_classic() +
  theme(
    axis.title = element_text(size = 14),
    axis.text = element_text(size = 12)
  ) +
  scale_x_datetime(
    breaks = break_points_N,
    date_labels = "%b %d, %H:%M"
  )



# Arrange all plots in a single layout
grid.arrange(plot_wd_S_wind, plot_p_S_wind, plot_ws_S_wind, nrow = 3, ncol = 1)
grid.arrange(plot_wd_N_wind, plot_p_N_wind, plot_ws_N_wind, nrow = 3, ncol = 1)



################################################################################
## AR model
################################################################################

# pacf for the wind power
pacf(X_train$p, main = "Power")


# AR1
model_AR1 <- arima(X_train$p , order = c(1,0,0), method = "CSS-ML")
model_AR1$coef
df_AR1_fit <- data.frame(
               t = X_test$t,
               obs = X_test$p,
               pred = rep(NA, length(X_test$p))
)

for(i in 2:nrow(X_test)){
  df_AR1_fit$pred[i] <- as.numeric(model_AR1$coef[2]) +
      as.numeric(model_AR1$coef[1])*X_test$p[i-1]
}

ggplot(data = df_AR1_fit[400:500, ], aes(x = t, y = obs))+
  geom_line()+
  geom_point(aes(x = t, y = pred))

resid_AR1 <- df_AR1_fit$obs[-1] - df_AR1_fit$pred[-1]
RMSE_AR1 <- sqrt(1/nrow(X_test) * sum(resid_AR1)^2)
```

```r
# AR2
model_AR2 <- arima(X_train$p , order = c(2,0,0), method = "CSS-ML")
model_AR2$coef
df_AR2_fit <- data.frame(
  t = X_test$t,
  obs = X_test$p,
  pred = rep(NA, length(X_test$p))
)

for(i in 3:nrow(X_test)){
  df_AR2_fit$pred[i] <- as.numeric(model_AR2$coef[3]) +
      as.numeric(model_AR2$coef[1])*X_test$p[i-1] +
      as.numeric(model_AR2$coef[2])*X_test$p[i-2]
}

ggplot(data = df_AR2_fit[400:500, ], aes(x = t, y = obs))+
  geom_line()+
  geom_point(aes(x = t, y = pred))

resid_AR2 <- df_AR2_fit$obs[-c(1,2)] - df_AR2_fit$pred[-c(1,2)]
RMSE_AR2 <- sqrt(1/nrow(X_test) * sum(resid_AR2)^2)
```

```r
# AR3
model_AR3 <- arima(X_train$p , order = c(3,0,0), method = "CSS-ML")
model_AR3$coef
df_AR3_fit <- data.frame(
  t = X_test$t,
  obs = X_test$p,
  pred = rep(NA, length(X_test$p))
)

for(i in 4:nrow(X_test)){
  df_AR3_fit$pred[i] <- as.numeric(model_AR3$coef[4]) +
      as.numeric(model_AR3$coef[1])*X_test$p[i-1] +
      as.numeric(model_AR3$coef[2])*X_test$p[i-2] +
      as.numeric(model_AR3$coef[3])*X_test$p[i-3]
}

ggplot(data = df_AR3_fit[400:500, ], aes(x = t, y = obs))+
  geom_line()+
  geom_point(aes(x = t, y = pred))

resid_AR3 <- df_AR3_fit$obs[-c(1,2,3)] - df_AR3_fit$pred[-c(1,2,3)]
RMSE_AR3 <- sqrt(1/nrow(X_test) * sum(resid_AR3)^2)
```

```r
# AR4
model_AR4 <- arima(X_train$p , order = c(4,0,0), method = "CSS-ML")
model_AR4$coef
df_AR4_fit <- data.frame(
  t = X_test$t,
  obs = X_test$p,
  pred = rep(NA, length(X_test$p))
)

for(i in 5:nrow(X_test)){
  df_AR4_fit$pred[i] <- as.numeric(model_AR4$coef[5]) +
      as.numeric(model_AR4$coef[1])*X_test$p[i-1] +
      as.numeric(model_AR4$coef[2])*X_test$p[i-2] +
      as.numeric(model_AR4$coef[3])*X_test$p[i-3] +
      as.numeric(model_AR4$coef[4])*X_test$p[i-4]
}

library(ggplot2)

# Example ggplot code
ggplot(data = df_AR4_fit[400:500, ], aes(x = t)) +
  geom_line(aes(y = obs, color = "Observed"), size = 1) +
  geom_point(aes(y = pred, color = "Predicted"), size = 2, shape = 16) +
  scale_color_manual(values = c("Observed" = "black", "Predicted" = "red"), name = "Legend")
      +
  labs(
    x = "Time",   # Customize x-axis title
    y = "Power",     # Customize y-axis title
  ) +

  scale_y_continuous(labels = scales::comma) +     # Adjust y-axis values format
  theme_classic() +
  theme(
    plot.title = element_text(hjust = 0.5), # Center the plot title
    legend.position = "right",           # Position the legend on the right
    legend.text = element_text(size = 12),      # Increase legend text size
    legend.title = element_text(size = 14),     # Increase legend title size
    axis.text = element_text(size = 12), # Restore axis text and adjust size
    axis.title = element_text(size = 14) # Adjust axis titles size
  )


resid_AR4 <- df_AR4_fit$obs[-c(1,2,3,4)] - df_AR4_fit$pred[-c(1,2,3,4)]
RMSE_AR4 <- sqrt(1/nrow(X_test) * sum(resid_AR4)^2)



# AR5
model_AR5 <- arima(X_train$p , order = c(5,0,0), method = "CSS-ML")
model_AR5$coef
df_AR5_fit <- data.frame(
```

```
  t = X_test$t,
  obs = X_test$p,
  pred = rep(NA, length(X_test$p))
)

for(i in 6:nrow(X_test)){
  df_AR5_fit$pred[i] <- as.numeric(model_AR5$coef[6]) +
      as.numeric(model_AR5$coef[1])*X_test$p[i-1] +
      as.numeric(model_AR5$coef[2])*X_test$p[i-2] +
      as.numeric(model_AR5$coef[3])*X_test$p[i-3] +
      as.numeric(model_AR5$coef[4])*X_test$p[i-4] +
      as.numeric(model_AR5$coef[5])*X_test$p[i-5]
}

ggplot(data = df_AR5_fit[400:500, ], aes(x = t, y = obs))+
  geom_line()+
  geom_point(aes(x = t, y = pred))

resid_AR5 <- df_AR5_fit$obs[-c(1,2,3,4,5)] - df_AR5_fit$pred[-c(1,2,3,4,5)]
RMSE_AR5 <- sqrt(1/nrow(X_test) * sum(resid_AR5)^2)



AIC(model_AR1)
AIC(model_AR2)
AIC(model_AR3)
AIC(model_AR4)
AIC(model_AR5)

BIC(model_AR1)
BIC(model_AR2)
BIC(model_AR3)
BIC(model_AR4)
BIC(model_AR5)

logLik(model_AR1)
logLik(model_AR2)
logLik(model_AR3)
logLik(model_AR4)
logLik(model_AR5)




################################################################################
## TARSO model with regime shift depending on wind direction
################################################################################

# Define a helper function to classify wind direction
classify_direction <- function(direction) {
  ifelse(direction >= 315 | direction < 45, "N",
```

```
  ifelse(direction >= 45 & direction < 135, "E",
  ifelse(direction >= 135 & direction < 225, "S", "W")))
}

# Apply the classification function
X_train$direction <- classify_direction(X_train$Wd1_shift)

# Subset data based on direction
X_N <- subset(X_train, direction == "N")
X_E <- subset(X_train, direction == "E")
X_S <- subset(X_train, direction == "S")
X_W <- subset(X_train, direction == "W")




# plot power vs. windspeed in each direction ###################################

# Define a common theme for consistency
custom_theme <- theme_classic() +
  theme(
    plot.title = element_text(hjust = 0.5, size = 18, face = "bold"), # Center title, larger
    axis.title = element_text(size = 16), # Larger axis labels
    axis.text = element_text(size = 14) # Larger axis values
  )

# Create individual ggplot objects with centered titles and larger text
plot_N <- ggplot(X_N, aes(x = Ws1_shift, y = p)) +
  geom_point(color = "blue") +
  labs(title = "North", x = "Wind Speed", y = "p") +
  custom_theme

plot_E <- ggplot(X_E, aes(x = Ws1_shift, y = p)) +
  geom_point(color = "blue") +
  labs(title = "East", x = "Wind Speed", y = "p") +
  custom_theme

plot_S <- ggplot(X_S, aes(x = Ws1_shift, y = p)) +
  geom_point(color = "blue") +
  labs(title = "South", x = "Wind Speed", y = "p") +
  custom_theme

plot_W <- ggplot(X_W, aes(x = Ws1_shift, y = p)) +
  geom_point(color = "blue") +
  labs(title = "West", x = "Wind Speed", y = "p") +
  custom_theme

# Combine the plots using grid.arrange
grid.arrange(plot_N, plot_E, plot_S, plot_W, nrow = 2)
```

```r
## Estimate the power curve as a scaled sigmoid function ########################
max_power_N <- max(X_N$p)
max_power_E <- max(X_E$p)
max_power_S <- max(X_S$p)
max_power_W <- max(X_W$p)


PC_N <- function(params, w) {
  a <- params[1]
  b <- params[2]
  max_power_N / (1 + exp(-a * (w - b)))
}

PC_E <- function(params, w) {
  a <- params[1]
  b <- params[2]
  max_power_E / (1 + exp(-a * (w - b)))
}

PC_S <- function(params, w) {
  a <- params[1]
  b <- params[2]
  max_power_S / (1 + exp(-a * (w - b)))
}

PC_W <- function(params, w) {
  a <- params[1]
  b <- params[2]
  max_power_W / (1 + exp(-a * (w - b)))
}


# Define objective function = sum of squared errors
SSE_PC_N <- function(params, w, p) {
  p_pred <- PC_N(params, w)
  sum((p - p_pred)^2)
}

SSE_PC_E <- function(params, w, p) {
  p_pred <- PC_E(params, w)
  sum((p - p_pred)^2)
}

SSE_PC_S <- function(params, w, p) {
  p_pred <- PC_S(params, w)
  sum((p - p_pred)^2)
}

SSE_PC_W <- function(params, w, p) {
  p_pred <- PC_W(params, w)
  sum((p - p_pred)^2)
```

```
}



# Initial guesses
init_params <- c(a=1, b=10)

PC_fit_N <- optim(par = init_params,
            fn = SSE_PC_N,
            w = X_N$Ws1_shift,
            p = X_N$p,
            method = "BFGS")
PC_fit_E <- optim(par = init_params,
                fn = SSE_PC_E,
                w = X_E$Ws1_shift,
                p = X_E$p,
                method = "BFGS")
PC_fit_S <- optim(par = init_params,
                fn = SSE_PC_S,
                w = X_S$Ws1_shift,
                p = X_S$p,
                method = "BFGS")
PC_fit_W <- optim(par = init_params,
                fn = SSE_PC_W,
                w = X_W$Ws1_shift,
                p = X_W$p,
                method = "BFGS")

PC_fit_N
PC_fit_E
PC_fit_S
PC_fit_W


# Define the power curves with the estimated parameters ########################
PC_N <- function(w) {
  a <- as.numeric(PC_fit_N$par[1])
  b <- as.numeric(PC_fit_N$par[2])
  max_power_N / (1 + exp(-a * (w - b)))
}

PC_E <- function(w) {
  a <- as.numeric(PC_fit_E$par[1])
  b <- as.numeric(PC_fit_E$par[2])
  max_power_E / (1 + exp(-a * (w - b)))
}

PC_S <- function(w) {
  a <- as.numeric(PC_fit_S$par[1])
  b <- as.numeric(PC_fit_S$par[2])
  max_power_S / (1 + exp(-a * (w - b)))
}
```

```
PC_W <- function(w) {
  a <- as.numeric(PC_fit_W$par[1])
  b <- as.numeric(PC_fit_W$par[2])
  max_power_W / (1 + exp(-a * (w - b)))
}


###### Plot power curves ######################
# Define sequence W.
wind_speed <- seq(0, 30, length.out = 970)

# Plot for North
plot_N <- ggplot(X_N, aes(x = Ws1_shift, y = p)) +
  geom_point(color = "lightgrey", alpha = 0.6) +
  geom_line(data = data.frame(wind_speed = wind_speed, p = PC_N(wind_speed)),
            aes(x = wind_speed, y = p), color = "#7CAE00", size = 1) + # Color for "N" from
                ggplot default
  labs(title = "North", x = "Wind Speed", y = "Power") +
  custom_theme

plot_N

# Plot for East
plot_E <- ggplot(X_E, aes(x = Ws1_shift, y = p)) +
  geom_point(color = "lightgrey", alpha = 0.6) +
  geom_line(data = data.frame(wind_speed = wind_speed, p = PC_E(wind_speed)),
            aes(x = wind_speed, y = p), color = "#F8766D", size = 1) + # Color for "E" from
                ggplot default
  labs(title = "East", x = "Wind Speed", y = "Power") +
  custom_theme

plot_E

# Plot for South
plot_S <- ggplot(X_S, aes(x = Ws1_shift, y = p)) +
  geom_point(color = "lightgrey", alpha = 0.6) +
  geom_line(data = data.frame(wind_speed = wind_speed, p = PC_S(wind_speed)),
            aes(x = wind_speed, y = p), color = "#00BFC4", size = 1) + # Color for "S" from
                ggplot default
  labs(title = "South", x = "Wind Speed", y = "Power") +
  custom_theme

plot_S

# Plot for West
plot_W <- ggplot(X_W, aes(x = Ws1_shift, y = p)) +
  geom_point(color = "lightgrey", alpha = 0.6) +
  geom_line(data = data.frame(wind_speed = wind_speed, p = PC_W(wind_speed)),
            aes(x = wind_speed, y = p), color = "#C77CFF", size = 1) + # Color for "W" from
                ggplot default
  labs(title = "West", x = "Wind Speed", y = "Power") +
```

```
  custom_theme

plot_W

# Combine the plots using grid.arrange
grid.arrange(plot_N, plot_E, plot_S, plot_W, nrow = 2)




# Define colors for the curves
curve_colors <- c("N" = "#F8766D", "E" = "#7CAE00", "S" = "#00BFC4", "W" = "#C77CFF")

# Create a single plot
combined_plot <- ggplot() +
  # Add data points for all directions in light gray
  geom_point(data = X_combined, aes(x = wind_speed, y = power),
             color = "lightgrey", size = 2, alpha = 0.6) +
  # Add fitted power curves for all directions
  geom_line(data = power_curves, aes(x = wind_speed, y = power, color = direction), size =
      1) +
  # Customize colors for the curves
  scale_color_manual(
    values = curve_colors,
    breaks = names(curve_colors)
  ) +
  labs(
    title = "Power Curves for Different Wind Directions",
    x = "Wind Speed",
    y = "Power",
    color = "Direction"
  ) +
  theme_classic() +
  theme(
    plot.title = element_text(hjust = 0.5, size = 18, face = "bold"),
    axis.title = element_text(size = 16),
    axis.text = element_text(size = 14),
    legend.title = element_text(size = 14),
    legend.text = element_text(size = 12)
  )

# Display the plot
print(combined_plot)


# fit model ##############################################################################



# define loss function for parameter estimation
lossTARSO<- function(theta, P, D, W) {
```

```r
#parameters for N
a11 <- theta[1]
a12 <- theta[2]
a13 <- theta[3]
b11 <- theta[4]

#parameters for E
a21 <- theta[5]
a22 <- theta[6]
a23 <- theta[7]
b21 <- theta[8]

#parameters for S
a31 <- theta[9]
a32 <- theta[10]
a33 <- theta[11]
b31 <- theta[12]

#parameters for W
a41 <- theta[13]
a42 <- theta[14]
a43 <- theta[15]
b41 <- theta[16]


n <- length(P)

# Compute prediction errors
prediction_errors <- numeric(n - 1)

for (i in 4:n) {
  if (D[i] >= 315 | D[i] < 45) {
    #   if(PC_N(W[i])<= 20.3){
    prediction_errors[i] <- P[i] - (a11*P[i-1] + a12*P[i-2] + a13*P[i-3] + b11*PC_N(W[i]))
    #   } else{
    #     prediction_errors[i] <- P[i] - 20
    #   }
    #
  } else if (D[i] >= 45 & D[i] < 135) {
    #   if(PC_E(W[i])<= 20.3){
    prediction_errors[i] <- P[i] - (a21*P[i-1] + a22*P[i-2] + a23*P[i-3] + b21*PC_E(W[i]))
    #   } else{
    #     prediction_errors[i] <- P[i] - 20
    #   }
    #
  } else if (D[i] >= 135 & D[i] < 225){
    #   if(PC_S(W[i])<= 20.3){
    prediction_errors[i] <- P[i] - (a31*P[i-1] + a32*P[i-2] + a33*P[i-3] + b31*PC_S(W[i]))
    #   } else {
    #     prediction_errors[i] <- P[i] - 20
    #   }
    #
```

```
    } else if (D[i] >= 225 & D[i] < 315){
      #
      #   if(PC_W(W[i])<= 20.3){
      prediction_errors[i] <- P[i] - (a41*P[i-1] + a42*P[i-2] + a43*P[i-3] + b41*PC_W(W[i])) }
    #   } else {
    #     prediction_errors[i] <- P[i] - 20
    #   }
    # }
  }

  # Return the sum of squared errors
  sum(prediction_errors^2)
}

# Initial guesses
init_params <- c(rep(1,1,16))

fit_TARSO <- optim(par = init_params,
                   fn = lossTARSO,
                   W = X_train[-1,]$Ws1_shift,
                   P = X_train[-1,]$p,
                   D = X_train[-1,]$Wd1_shift,
                   method = "BFGS")

fit_TARSO$par

# 1 hour ahead forecast ##########################################################
#
model_TARSO <- function(theta, D, W, P_lags) {

  #parameters for N
  a11 <- theta[1]
  a12 <- theta[2]
  a13 <- theta[3]
  b11 <- theta[4]

  #parameters for E
  a21 <- theta[5]
  a22 <- theta[6]
  a23 <- theta[7]
  b21 <- theta[8]

  #parameters for S
  a31 <- theta[9]
  a32 <- theta[10]
  a33 <- theta[11]
  b31 <- theta[12]

  #parameters for W
  a41 <- theta[13]
  a42 <- theta[14]
  a43 <- theta[15]
```

```
  b41 <- theta[16]


   if (D >= 315 | D < 45) {

        P <- a11*P_lags[1] + a12*P_lags[2] + a13*P_lags[3] + b11*PC_N(W)

   } else if (D >= 45 & D < 135) {
        P <- a21*P_lags[1] + a22*P_lags[2] + a23*P_lags[3] + b21*PC_E(W)

     } else if (D >= 135 & D < 225){

        P <- a31*P_lags[1] + a32*P_lags[2] + a33*P_lags[3] + b31*PC_S(W)

     }else if (D >= 225 & D < 315){

        P <- a41*P_lags[1] + a42*P_lags[2] + a43*P_lags[3] + b41*PC_W(W)

     }
  # Return estimated power production
  P
}


df_TARSO_fit <- data.frame(
  t = X_test$t,
  obs = X_test$p,
  pred = rep(NA, length(X_test$p))
)


# doing 1 ahed prediction
for (i in 4:nrow(X_test)){
  df_TARSO_fit$pred[i] <- model_TARSO(c(as.numeric(fit_TARSO$par)),
                                      X_test$Wd1_shift[i],
                                      X_test$Ws1_shift[i],
                                      c(X_test$p[i-1], X_test$p[i-2], X_test$p[i-3], X_test$p[i-4]))

}

ggplot(data = df_TARSO_fit[400:500,], aes(x = t, y = obs))+
  geom_line() +
  geom_point(aes(x = t, y = pred))




# look at residuals ############################################################
par(mfrow = c(1,1))
resid <- df_TARSO_fit$obs[4:1000] - df_TARSO_fit$pred[4:1000]
```

```r
plot(df_TARSO_fit$pred[4:1000], resid)
plot(df_TARSO_fit$t[4:1000], resid)
abline(h = 0)
qqnorm(resid)
qqline(resid)

# evaluate the model using root mean squared error
RMSE_TARSO <- sqrt(1/length(resid) * sum(resid)^2)
RMSE_TARSO
```

```r
################################################################################
## Include diurnal period in model
################################################################################


# define loss function for parameter estimation
lossDirunal<- function(theta, P, D, W, h) {

  #parameters for N
  a11 <- theta[1]
  a12 <- theta[2]
  a13 <- theta[3]
  b11 <- theta[4]

  #parameters for E
  a21 <- theta[5]
  a22 <- theta[6]
  a23 <- theta[7]
  b21 <- theta[8]

  #parameters for S
  a31 <- theta[9]
  a32 <- theta[10]
  a33 <- theta[11]
  b31 <- theta[12]

  #parameters for W
  a41 <- theta[13]
  a42 <- theta[14]
  a43 <- theta[15]
  b41 <- theta[16]


  n <- length(P)

  # Compute prediction errors
```

```
prediction_errors <- numeric(n - 1)

for (i in 4:n) {
  # print(i)
  if (D[i] >= 315 | D[i] < 45) {
    #  print(11)
    #   if(PC_N(W[i])<= 20.3){
    prediction_errors[i] <- P[i] - (a11*P[i-1] + a12*P[i-2] + a13*P[i-3] +
        b11*PC_N(W[i])*(1 + sin(2*pi*h[i]/24) + cos(2*pi*h[i]/24)))
    #   } else{
    #     prediction_errors[i] <- P[i] - 20
    #   }
    #
  } else if (D[i] >= 45 & D[i] < 135) {
    #  print(22)
    #   if(PC_E(W[i])<= 20.3){
    #print(prediction_errors[i])
    prediction_errors[i] <- P[i] - (a21*P[i-1] + a22*P[i-2] + a23*P[i-3] +
        b21*PC_E(W[i])*(1 + sin(2*pi*h[i]/24) + cos(2*pi*h[i]/24)))

    #   } else{
    #     prediction_errors[i] <- P[i] - 20
    #   }
    #
  } else if (D[i] >= 135 & D[i] < 225){
    # print(33)
    # print("P[i]")
    # print(P[i])
    # print("P[i-1]")
    # print(P[i-1])
    # print("P[i-2]")
    # print(P[i-2])
    # print("P[i-3]")
    # print(P[i-3])
    # print("PC_S(W[i])")
    # print(PC_S(W[i]))
    # print("sin")
    # print(sin(2*pi*h[i]/24))
    # print(2*pi*h[i]/24)
    # print(h[i])
    # print("cos")
    # print(cos(2*pi*h[i]/24))
    #   if(PC_S(W[i])<= 20.3){
    prediction_errors[i] <- P[i] - (a31*P[i-1] + a32*P[i-2] + a33*P[i-3] +
        b31*PC_S(W[i])*(1 + sin(2*pi*h[i]/24) + cos(2*pi*h[i]/24)))
    #   } else {
    #     prediction_errors[i] <- P[i] - 20
    #   }
    #
  } else if (D[i] >= 225 & D[i] < 315){
    # print(44)
    #
```

```
      #   if(PC_W(W[i])<= 20.3){
      prediction_errors[i] <- P[i] - (a41*P[i-1] + a42*P[i-2] + a43*P[i-3] +
          b41*PC_W(W[i])*(1 + sin(2*pi*h[i]/24) + cos(2*pi*h[i]/24)))
      #   } else {
      #     prediction_errors[i] <- P[i] - 20
    }
  }

  # Return the sum of squared errors
  sum(prediction_errors^2)
}



# Initial guesses
init_params <- c(rep(1,1,16))

fit_Diurnal <- optim(par = init_params,
                     fn = lossDirunal,
                     W = X_train[-1,]$Ws1_shift,
                     P = X_train[-1,]$p,
                     D = X_train[-1,]$Wd1_shift,
                     h = X_train[-1,]$hour,
                     method = "BFGS")

fit_Diurnal$par

# 1 hour ahead forecast model ######################################################


model_Diurnal <- function(theta, D, W, P_lags, h) {

  #parameters for N
  a11 <- theta[1]
  a12 <- theta[2]
  a13 <- theta[3]
  b11 <- theta[4]

  #parameters for E
  a21 <- theta[5]
  a22 <- theta[6]
  a23 <- theta[7]
  b21 <- theta[8]

  #parameters for S
  a31 <- theta[9]
  a32 <- theta[10]
  a33 <- theta[11]
  b31 <- theta[12]

  #parameters for W
  a41 <- theta[13]
```

```
  a42 <- theta[14]
  a43 <- theta[15]
  b41 <- theta[16]


  if (D >= 315 | D < 45) {
    #     if(PC_N(W) <= 20.3){
    P <- a11*P_lags[1] + a12*P_lags[2] + a13*P_lags[3] + b11*PC_N(W)*(1 + sin(2*pi*h/24) +
        cos(2*pi*h/24))
    #     } else {
    #       P <- 20.3
    #     }
  } else if (D >= 45 & D < 135) {
    #     if(PC_E(W)<= 20.3){
    P <- a21*P_lags[1] + a22*P_lags[2] + a23*P_lags[3] + b21*PC_E(W)*(1 + sin(2*pi*h/24) +
        cos(2*pi*h/24))
    #     } else{
    #       p <- 20.3
    #     }
    #
  } else if (D >= 135 & D < 225){
    #     if( PC_S(W)<= 20.3){
    P <- a31*P_lags[1] + a32*P_lags[2] + a33*P_lags[3] + b31*PC_S(W)*(1 + sin(2*pi*h/24) +
        cos(2*pi*h/24))
    #     } else{
    #       P <- 20.3
    #     }
    #
  }else if (D >= 225 & D < 315){
    #     if(PC_W(W)<= 20.3){
    P <- a41*P_lags[1] + a42*P_lags[2] + a43*P_lags[3] + b41*PC_W(W)*(1 + sin(2*pi*h/24) +
        cos(2*pi*h/24))
    #     } else{
    #       P<- 20.3
    #     }
  }
  # Return estimated power production
  P
}



df_Diurnal_fit <- data.frame(
  t = X_test$t,
  obs = X_test$p,
  pred = rep(NA, length(X_test$p))
)


# doing 1 ahed prediction
for (i in 4:nrow(X_test)){
  df_Diurnal_fit$pred[i] <- model_Diurnal(c(as.numeric(fit_Diurnal$par)),
```

```
                                    X_test$Wd1_shift[i],
                                    X_test$Ws1_shift[i],
                                    c(X_test$p[i-1], X_test$p[i-2], X_test$p[i-3]),
                                    X_test$hour[i])

}



ggplot(data = df_Diurnal_fit[400:500,], aes(x = t, y = obs))+
  geom_line() +
  geom_point(aes(x = t, y = pred))







# look at residuals ################################################################
par(mfrow = c(1,1))
resid <- df_Diurnal_fit$obs[-c(1,2,3,4,5)] - df_Diurnal_fit$pred[-c(1,2,3,4,5)]
plot(df_Diurnal_fit$pred[-c(1,2,3,4,5)], resid, xlab = "fitted")
abline(h = 0, col = "red")
plot(df_Diurnal_fit$t[-c(1,2,3,4,5)], resid)

qqnorm(resid)
qqline(resid)

# evaluate the model using root mean squared error
RMSE_Diurnal <- sqrt(1/length(resid) * sum(resid)^2)
RMSE_Diurnal









##############################################################################
## 2 hour forecast
##############################################################################

# Prepare data frame
df_2hf <- data.frame(
  t = X$t,
  p = X$p,
  Ws2_shift = X$Ws2_shift,
  Wd2_shift = X$Wd2_shift,
  h = X$hour
)
df_2hf <- df_2hf[-c(1,2),]
```

```r
# remove every 2nd observation, so we only get 2 hour points
df_2hf <- df_2hf[seq(1, nrow(df_2hf), by = 2), ]

df_2hf_train <- df_2hf[1:1500,]
df_2hf_test <- df_2hf[4500:5000,]

# if AR ---------------

pacf(df_2hf_train$p)



# redefine power curve --------------------------------


# Define a helper function to classify wind direction
classify_direction <- function(direction) {
  ifelse(direction >= 315 | direction < 45, "N",
         ifelse(direction >= 45 & direction < 135, "E",
                ifelse(direction >= 135 & direction < 225, "S", "W")))
}

# Apply the classification function
df_2hf_train$direction <- classify_direction(df_2hf_train$Wd2_shift)

# Subset data based on direction
X_N <- subset(df_2hf_train, direction == "N")
X_E <- subset(df_2hf_train, direction == "E")
X_S <- subset(df_2hf_train, direction == "S")
X_W <- subset(df_2hf_train, direction == "W")



# plot power vs. windspeed in each direction ###################################

# Define a common theme for consistency
custom_theme <- theme_classic() +
  theme(
    plot.title = element_text(hjust = 0.5, size = 18, face = "bold"), # Center title, larger
    axis.title = element_text(size = 16), # Larger axis labels
    axis.text = element_text(size = 14) # Larger axis values
  )

# Create individual ggplot objects with centered titles and larger text
plot_N <- ggplot(X_N, aes(x = Ws2_shift, y = p)) +
  geom_point(color = "blue") +
  labs(title = "North", x = "Wind Speed", y = "p") +
  custom_theme
```

```r
plot_E <- ggplot(X_E, aes(x = Ws2_shift, y = p)) +
  geom_point(color = "blue") +
  labs(title = "East", x = "Wind Speed", y = "p") +
  custom_theme

plot_S <- ggplot(X_S, aes(x = Ws2_shift, y = p)) +
  geom_point(color = "blue") +
  labs(title = "South", x = "Wind Speed", y = "p") +
  custom_theme

plot_W <- ggplot(X_W, aes(x = Ws2_shift, y = p)) +
  geom_point(color = "blue") +
  labs(title = "West", x = "Wind Speed", y = "p") +
  custom_theme

# Combine the plots using grid.arrange
grid.arrange(plot_N, plot_E, plot_S, plot_W, nrow = 2)
```

```r
## Estimate the power curve as a scaled sigmoid function ########################
max_power_N <- max(X_N$p)
max_power_E <- max(X_E$p)
max_power_S <- max(X_S$p)
max_power_W <- max(X_W$p)


PC_N <- function(params, w) {
  a <- params[1]
  b <- params[2]
  max_power_N / (1 + exp(-a * (w - b)))
}

PC_E <- function(params, w) {
  a <- params[1]
  b <- params[2]
  max_power_E / (1 + exp(-a * (w - b)))
}

PC_S <- function(params, w) {
  a <- params[1]
  b <- params[2]
  max_power_S / (1 + exp(-a * (w - b)))
}

PC_W <- function(params, w) {
  a <- params[1]
  b <- params[2]
  max_power_W / (1 + exp(-a * (w - b)))
}
```

```r
# Define objective function = sum of squared errors
SSE_PC_N <- function(params, w, p) {
  p_pred <- PC_N(params, w)
  sum((p - p_pred)^2)
}

SSE_PC_E <- function(params, w, p) {
  p_pred <- PC_E(params, w)
  sum((p - p_pred)^2)
}

SSE_PC_S <- function(params, w, p) {
  p_pred <- PC_S(params, w)
  sum((p - p_pred)^2)
}

SSE_PC_W <- function(params, w, p) {
  p_pred <- PC_W(params, w)
  sum((p - p_pred)^2)
}



# Initial guesses
init_params <- c(a=1, b=10)

PC_fit_N <- optim(par = init_params,
                  fn = SSE_PC_N,
                  w = X_N$Ws2_shift,
                  p = X_N$p,
                  method = "BFGS")
PC_fit_E <- optim(par = init_params,
                  fn = SSE_PC_E,
                  w = X_E$Ws2_shift,
                  p = X_E$p,
                  method = "BFGS")
PC_fit_S <- optim(par = init_params,
                  fn = SSE_PC_S,
                  w = X_S$Ws2_shift,
                  p = X_S$p,
                  method = "BFGS")
PC_fit_W <- optim(par = init_params,
                  fn = SSE_PC_W,
                  w = X_W$Ws2_shift,
                  p = X_W$p,
                  method = "BFGS")

PC_fit_N$par
PC_fit_E$par
PC_fit_S$par
```

```
PC_fit_W$par


# Define the power curves with the estimated parameters ########################
PC_N <- function(w) {
  a <- as.numeric(PC_fit_N$par[1])
  b <- as.numeric(PC_fit_N$par[2])
  max_power_N / (1 + exp(-a * (w - b)))
}

PC_E <- function(w) {
  a <- as.numeric(PC_fit_E$par[1])
  b <- as.numeric(PC_fit_E$par[2])
  max_power_E / (1 + exp(-a * (w - b)))
}

PC_S <- function(w) {
  a <- as.numeric(PC_fit_S$par[1])
  b <- as.numeric(PC_fit_S$par[2])
  max_power_S / (1 + exp(-a * (w - b)))
}

PC_W <- function(w) {
  a <- as.numeric(PC_fit_W$par[1])
  b <- as.numeric(PC_fit_W$par[2])
  max_power_W / (1 + exp(-a * (w - b)))
}




init_params <- rep(1,1,16)

fit_Diurnal_2hf <- optim(par = init_params,
                         fn = lossDirunal,
                         W = df_2hf_train$Ws2_shift,
                         P = df_2hf_train$p,
                         D = df_2hf_train$Wd2_shift,
                         h = df_2hf_train$h,
                         method = "BFGS")


df_Diurnal_2hf_fit <- data.frame(
  t = df_2hf_test$t,
  obs = df_2hf_test$p,
  pred = rep(NA, length(df_2hf_test$p))
)


# doing 1 ahed prediction
for (i in 4:nrow(df_Diurnal_2hf_fit)){
  df_Diurnal_2hf_fit$pred[i] <- model_Diurnal(c(as.numeric(fit_Diurnal_2hf$par)),
```

```
                                          df_2hf_test$Wd2_shift[i],
                                          df_2hf_test$Ws2_shift[i],
                                          c(df_2hf_test$p[i-1], df_2hf_test$p[i-2],
                                              df_2hf_test$p[i-3]),
                                          df_2hf_test$h)
  print(i)
}

ggplot(data = df_Diurnal_2hf_fit[100:130,], aes(x = t, y = obs))+
  geom_line() +
  geom_point(aes(x = t, y = pred))



# look at residuals ############################################################
par(mfrow = c(1,1))
resid <- df_Diurnal_2hf_fit$obs[4:nrow(df_Diurnal_2hf_fit)] -
    df_Diurnal_2hf_fit$pred[4:nrow(df_Diurnal_2hf_fit)]
plot(df_Diurnal_2hf_fit$pred[4:nrow(df_Diurnal_2hf_fit)], resid)
plot(df_Diurnal_2hf_fit$t[4:nrow(df_Diurnal_2hf_fit)], resid)
abline(h = 0, col = "red")
qqnorm(resid)
qqline(resid)

# evaluate the model using root mean squared error
RMSE_Diurnal_2hf <- sqrt(1/length(resid) * sum(resid)^2)
RMSE_Diurnal_2hf
```

```
init_params <- rep(1,1,16)

fit_Diurnal_2hf <- optim(par = init_params,
                      fn = lossDirunal,
                      W = df_2hf_train$Ws2_shift,
                      P = df_2hf_train$p,
                      D = df_2hf_train$Wd2_shift,
                      h = df_2hf_train$h,
                      method = "BFGS")


df_Diurnal_2hf_fit <- data.frame(
  t = df_2hf_test$t,
  obs = df_2hf_test$p,
  pred = rep(NA, length(df_2hf_test$p))
)


# doing 1 ahed prediction
for (i in 4:nrow(df_Diurnal_2hf_fit)){
  df_Diurnal_2hf_fit$pred[i] <- model_Diurnal(c(as.numeric(fit_Diurnal_2hf$par)),
                                      df_2hf_test$Wd2_shift[i],
                                      df_2hf_test$Ws2_shift[i],
                                      c(df_2hf_test$p[i-1], df_2hf_test$p[i-2],
                                          df_2hf_test$p[i-3]),
                                      df_2hf_test$h)
  print(i)
}

ggplot(data = df_Diurnal_2hf_fit[100:130,], aes(x = t, y = obs))+
  geom_line() +
  geom_point(aes(x = t, y = pred))




# look at residuals ############################################################
par(mfrow = c(1,1))
resid <- df_Diurnal_2hf_fit$obs[4:nrow(df_Diurnal_2hf_fit)] -
    df_Diurnal_2hf_fit$pred[4:nrow(df_Diurnal_2hf_fit)]
plot(df_Diurnal_2hf_fit$pred[4:nrow(df_Diurnal_2hf_fit)], resid)
plot(df_Diurnal_2hf_fit$t[4:nrow(df_Diurnal_2hf_fit)], resid)
abline(h = 0, col = "red")
qqnorm(resid)
qqline(resid)

# evaluate the model using root mean squared error
```

```
RMSE_Diurnal_2hf <- sqrt(1/length(resid) * sum(resid)^2)
RMSE_Diurnal_2hf
```

```
################################################################################
## 3 hour forecast
################################################################################

# Prepare data frame
df_3hf <- data.frame(
  t = X$t,
  p = X$p,
  Ws3_shift = X$Ws3_shift,
  Wd3_shift = X$Wd3_shift,
  h = X$hour
)
df_3hf <- df_3hf[-c(1,2,3),]

# remove every 2nd observation, so we only get 2 hour points
df_3hf <- df_3hf[seq(1, nrow(df_3hf), by = 3), ]

df_3hf_train <- df_3hf[1:2000,]
df_3hf_test <- df_3hf[3000:3300,]



# initial AR
pacf(df_3hf_train$p, main = " ")



################################################################################
# redefine power curve



# redefine power curve -------------------------------
```

```r
# Define a helper function to classify wind direction
classify_direction <- function(direction) {
  ifelse(direction >= 315 | direction < 45, "N",
         ifelse(direction >= 45 & direction < 135, "E",
                ifelse(direction >= 135 & direction < 225, "S", "W")))
}

# Apply the classification function
df_3hf_train$direction <- classify_direction(df_3hf_train$Wd3_shift)

# Subset data based on direction
X_N <- subset(df_3hf_train, direction == "N")
X_E <- subset(df_3hf_train, direction == "E")
X_S <- subset(df_3hf_train, direction == "S")
X_W <- subset(df_3hf_train, direction == "W")




## Estimate the power curve as a scaled sigmoid function ########################
max_power_N <- max(X_N$p)
max_power_E <- max(X_E$p)
max_power_S <- max(X_S$p)
max_power_W <- max(X_W$p)


PC_N <- function(params, w) {
  a <- params[1]
  b <- params[2]
  max_power_N / (1 + exp(-a * (w - b)))
}

PC_E <- function(params, w) {
  a <- params[1]
  b <- params[2]
  max_power_E / (1 + exp(-a * (w - b)))
}

PC_S <- function(params, w) {
  a <- params[1]
  b <- params[2]
  max_power_S / (1 + exp(-a * (w - b)))
}

PC_W <- function(params, w) {
  a <- params[1]
  b <- params[2]
  max_power_W / (1 + exp(-a * (w - b)))
}


# Define objective function = sum of squared errors
```

```r
SSE_PC_N <- function(params, w, p) {
  p_pred <- PC_N(params, w)
  sum((p - p_pred)^2)
}

SSE_PC_E <- function(params, w, p) {
  p_pred <- PC_E(params, w)
  sum((p - p_pred)^2)
}

SSE_PC_S <- function(params, w, p) {
  p_pred <- PC_S(params, w)
  sum((p - p_pred)^2)
}

SSE_PC_W <- function(params, w, p) {
  p_pred <- PC_W(params, w)
  sum((p - p_pred)^2)
}



# Initial guesses
init_params <- c(a=1, b=10)

PC_fit_N <- optim(par = init_params,
                  fn = SSE_PC_N,
                  w = X_N$Ws3_shift,
                  p = X_N$p,
                  method = "BFGS")
PC_fit_E <- optim(par = init_params,
                  fn = SSE_PC_E,
                  w = X_E$Ws3_shift,
                  p = X_E$p,
                  method = "BFGS")
PC_fit_S <- optim(par = init_params,
                  fn = SSE_PC_S,
                  w = X_S$Ws3_shift,
                  p = X_S$p,
                  method = "BFGS")
PC_fit_W <- optim(par = init_params,
                  fn = SSE_PC_W,
                  w = X_W$Ws3_shift,
                  p = X_W$p,
                  method = "BFGS")

PC_fit_N$par
PC_fit_E$par
PC_fit_S$par
PC_fit_W$par
```

```r
# Define the power curves with the estimated parameters ########################
PC_N <- function(w) {
  a <- as.numeric(PC_fit_N$par[1])
  b <- as.numeric(PC_fit_N$par[2])
  max_power_N / (1 + exp(-a * (w - b)))
}

PC_E <- function(w) {
  a <- as.numeric(PC_fit_E$par[1])
  b <- as.numeric(PC_fit_E$par[2])
  max_power_E / (1 + exp(-a * (w - b)))
}

PC_S <- function(w) {
  a <- as.numeric(PC_fit_S$par[1])
  b <- as.numeric(PC_fit_S$par[2])
  max_power_S / (1 + exp(-a * (w - b)))
}

PC_W <- function(w) {
  a <- as.numeric(PC_fit_W$par[1])
  b <- as.numeric(PC_fit_W$par[2])
  max_power_W / (1 + exp(-a * (w - b)))
}
```

```r
init_params <- rep(1,1,16)

fit_Diurnal_3hf <- optim(par = init_params,
                         fn = lossDirunal,
                         W = df_3hf_train$Ws3_shift,
                         P = df_3hf_train$p,
                         D = df_3hf_train$Wd3_shift,
                         h = df_3hf_train$h,
                         method = "BFGS")


fit_Diurnal_3hf$par
```

```r
df_Diurnal_3hf_fit <- data.frame(
  t = df_3hf_test$t,
  obs = df_3hf_test$p,
  pred = rep(NA, length(df_3hf_test$p))
)


# doing 1 ahed prediction
for (i in 4:nrow(df_3hf_test)){
  df_Diurnal_3hf_fit$pred[i] <- model_Diurnal(c(as.numeric(fit_Diurnal_3hf$par)),
                                              df_3hf_test$Wd3_shift[i],
                                              df_3hf_test$Ws3_shift[i],
                                              c(df_3hf_test$p[i-1], df_3hf_test$p[i-2],
                                                df_3hf_test$p[i-3]),
                                              df_3hf_test$h)
  print(i)
}

ggplot(data = df_Diurnal_3hf_fit[100:200,], aes(x = t, y = obs))+
  geom_line() +
  geom_point(aes(x = t, y = pred))

ggplot(data = df_Diurnal_3hf_fit[100:150,], aes(x = t)) +
  geom_line(aes(y = obs, color = "Observed"), size = 1) +
  geom_point(aes(y = pred, color = "Predicted"), size = 2, shape = 16) +
  scale_color_manual(values = c("Observed" = "black", "Predicted" = "red"), name = "Legend")
      +
  labs(
    x = "Time",   # Customize x-axis title
    y = "Power",     # Customize y-axis title
  ) +

  scale_y_continuous(labels = scales::comma) +     # Adjust y-axis values format
  theme_classic() +
  theme(
    plot.title = element_text(hjust = 0.5), # Center the plot title
    legend.position = "right",           # Position the legend on the right
    legend.text = element_text(size = 12),      # Increase legend text size
    legend.title = element_text(size = 14),     # Increase legend title size
    axis.text = element_text(size = 12), # Restore axis text and adjust size
    axis.title = element_text(size = 14) # Adjust axis titles size
  )




# look at residuals ############################################################
par(mfrow = c(1,1))
```

```
resid <- df_Diurnal_3hf_fit$obs[4:nrow(df_Diurnal_3hf_fit)] -
    df_Diurnal_3hf_fit$pred[4:nrow(df_Diurnal_3hf_fit)]
plot(df_Diurnal_3hf_fit$pred[4:nrow(df_Diurnal_3hf_fit)], resid)
plot(df_Diurnal_3hf_fit$t[4:nrow(df_Diurnal_3hf_fit)], resid)
abline(h = 0, col = "red")
qqnorm(resid)
qqline(resid)

# evaluate the model using root mean squared error
RMSE_Diurnal_3hf <- sqrt(1/length(resid) * sum(resid)^2)
RMSE_Diurnal_3hf
```